

Software Engineering Practice

Analysis and Design of a PoS System

Name: Taoliyuan

NUIST ID: 202383890020

SETU ID: 20109990

Email: welt799@163.com

GitHub URL: https://github.com/Welt-Tao/ProjectPos_2026

Published; main and taoliyuan/development verified on 15 October 2026

Demo Video URL: Pending student recording and publication

15 October 2026

Contents

1 Project overview	3
2 Agile process and iteration plan	4
3 Product backlog	5
4 Burndown and actual evidence	6
5 Requirements and business rules	7
6 Use case model	8
7 UC1 Process Sale	9
8 UC1 alternatives and system events	10
9 UC2 Handle Returns	11
10 UC2 alternatives and system events	12
11 Domain model	13
12 Logical architecture	14
13 Object design and responsibilities	15
14 Sale interaction design	16
15 Return interaction design	17
16 Persistence and API	18
17 Working cash checkout	19
18 Working returns and history	20
19 Verification results	21
20 Version control and delivery	22
21 Evaluation and references	23
Appendix A Application controller	24

This report presents the individual Market POS project, its Agile plan, use-case analysis, object-oriented design and implementation evidence. The accompanying repository contains complete source code and tests. The presentation and script support a separate narrated demonstration.

1 Project overview

Market POS is a local supermarket checkout application with two core use cases: Process Sale and Handle Returns. The cashier selects products, receives cash and creates a persistent receipt. A return refers to that receipt, calculates a refund at the original unit price and restores eligible items to inventory.

Taoliyuan completes the project individually, with permission to use an individual arrangement. The software and all submission materials use English. The submission deadline is 24 November 2026.

Objectives and scope

1. Deliver a usable cash checkout with a running basket total, payment validation, change calculation and receipt.

2. Support partial or full returns of unreturned items on an existing receipt. Prevent over-refunding and restore stock.

3. Keep transaction records and stock changes consistent after errors, restarts and repeated requests.

4. Trace the assignment requirements through use cases, UML, source files and verification evidence.

Explicit assumptions

EUR is the demonstration currency and displayed prices include applicable tax. Only whole item quantities are supported. Returned goods are assumed suitable for resale. The cashier performs the physical cash exchange. There is no return time limit in this version. Missing-receipt and damaged-goods returns are outside scope.

Technology and limits

Python 3.10 or newer supplies the HTTP server, SQLite interface and test framework. HTML, CSS and JavaScript provide a browser interface. No third-party Python packages are needed to run the system. The server listens only on localhost. This is a coursework implementation, not a production retail deployment.

2 Agile process and iteration plan

The project uses a Scrum-inspired individual workflow. One person maintains priorities, implements the increment and reviews its acceptance criteria. It does not claim the role separation or collaboration of a full Scrum Team. AI assistance supported code and document preparation; the student retains responsibility for review, understanding and presentation.

Sprint	Planned timebox	Goal and exit criterion
1	15–17 Oct 2026	Cash sale, receipt and stock. EUR 8.20 sale with EUR 10 cash gives EUR 1.80 change.
2	18–20 Oct 2026	Receipt lookup, returns and history. One milk unit refunds EUR 2.50 with correct stock.
3	21–23 Oct 2026	UML, report, presentation, remote repository and recorded demo. Links and evidence are verified.

Each timebox is three days. Implementation can finish early, but early completion is recorded with its actual date. The planned dates do not represent days of work already elapsed. Daily actual burndown values remain blank until observed.

Working routine

At the start of a timebox, select the highest-value stories with clear acceptance criteria. During work, keep an ordered task list and record completed checks. At review, compare the increment to the criteria and update the backlog. A retrospective records a concrete improvement rather than inventing meeting attendance or stakeholder feedback.

Definition of done

A software story is done when its acceptance criteria pass, relevant tests pass, the user interface is exercised, and documentation matches behavior. The release is done only when source branches are pushed, the student has rehearsed and recorded an English demonstration, and both final URLs are present in the report.

Feedback and risk management

Automated checks and the browser walkthrough provide technical feedback. Student acceptance and assessor feedback are separate and remain unclaimed until received. The main delivery risks are authentication during publication, an unreadable recording and inconsistent model/code terminology. Keep publication time available before 24 November and rehearse against the fixed demonstration scenario.

3 Product backlog

Story points are relative estimates, not measured hours. Must stories support required use cases and evidence; the history view is a Should story. Full user-story wording and editable status fields are in Agile_Plan.xlsx.

ID	Requirement and acceptance summary	Priority	Points	Sprint
US01	Two milk units and one bread show EUR 8.20.	Must	3	1
US02	EUR 10.00 tendered for EUR 8.20 yields EUR 1.80 change. Insufficient cash fails.	Must	5	1
US03	Receipt contains a unique ID, original prices, quantities, total and payment.	Must	3	1
US04	Sales reduce stock. Rejected and concurrent excess sales cannot overdraw stock.	Must	5	1
US05	Lookup shows purchased, already returned and remaining quantities.	Must	3	2
US06	One milk unit refunds EUR 2.50 using the original price and restores one unit.	Must	5	2
US07	Retries have one effect. Cumulative returns never exceed quantities sold.	Must	5	2
US08	EUR 8.20 sales less EUR 2.50 refunds gives EUR 5.70 net.	Should	2	2
US09	Both use cases, UML models, report, workbook and presentation are readable and consistent.	Must	5	3
US10	Both branches are available remotely and a published English presentation video is linked.	Must	3	3

The initial backlog totals 39 points. The acceptance-verified implementation accounts for 31 points. After artifact review, US09 is also complete: 36 points are done and the 3-point delivery story remains open for the student video. The chart below retains the initial implementation checkpoints.

4 Burndown and actual evidence

The workbook contains one ideal three-day burndown for each sprint and a separate actual checkpoint chart. The ideal series is initial sprint points multiplied by one minus elapsed days divided by three. Actual checkpoint values decrease only after the relevant acceptance evidence exists.

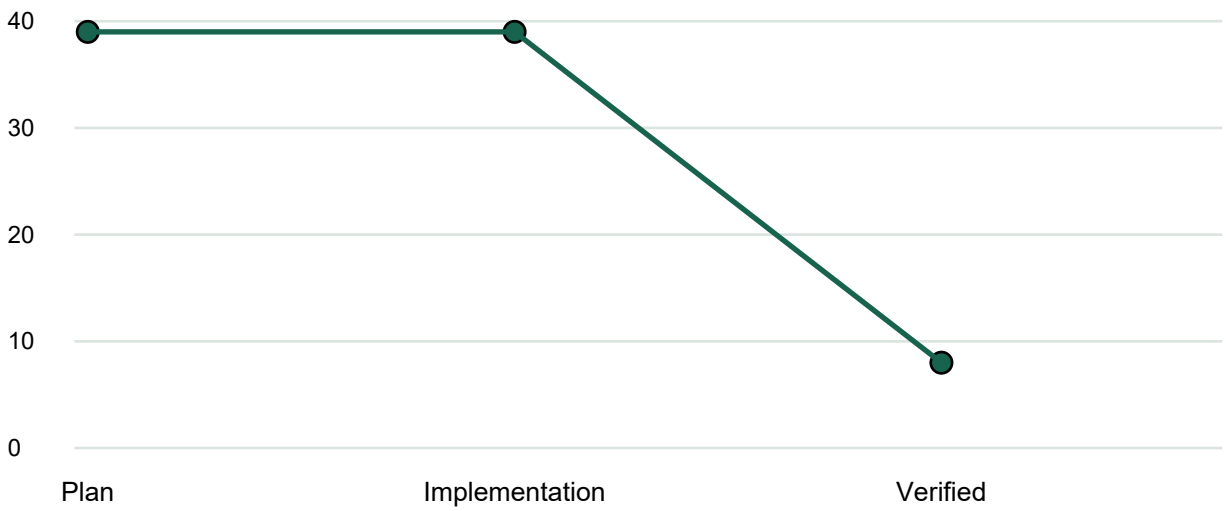


Figure 1. Actual remaining story points at observed checkpoints on 15 October 2026.

Observed time	Commit	Remaining	Evidence
22:02 +08:00	f2811d3	39	Plan baseline recorded
22:02 +08:00	b44e312	39	Implementation exists but acceptance is not yet recorded
22:08 +08:00	314bd47	8	22 automated tests pass and both workflows exercised

Review and next improvement

The successful sale and partial return matched the expected amounts and inventory. Error-path tests found no committed changes on rejection. The next review should be performed by the student, with attention to understandable button labels and the ability to explain the original-price refund rule. No stakeholder interview or multi-day meeting record is implied by these checkpoints.

5 Requirements and business rules

ID	Rule	Verification
BR01	Money is stored as integer cents. Cash input accepts at most two decimal places.	Money validation tests
BR02	A sale contains at least one known product and positive whole quantities.	Empty, unknown and quantity tests
BR03	Cash must cover the total. Stock must cover the quantity at commit time.	Insufficient cash and stock tests
BR04	Sale price snapshots remain immutable after completion.	Original-price refund test
BR05	A return needs a valid receipt and a reason of 3 to 200 characters.	Missing receipt and reason tests
BR06	Cumulative returned quantity cannot exceed purchased quantity.	Full, excess and concurrent return tests
BR07	Receipt rows, return rows and stock changes commit atomically.	Rollback and concurrency tests
BR08	Retrying the same token and payload produces the same transaction.	Sale/return idempotency tests

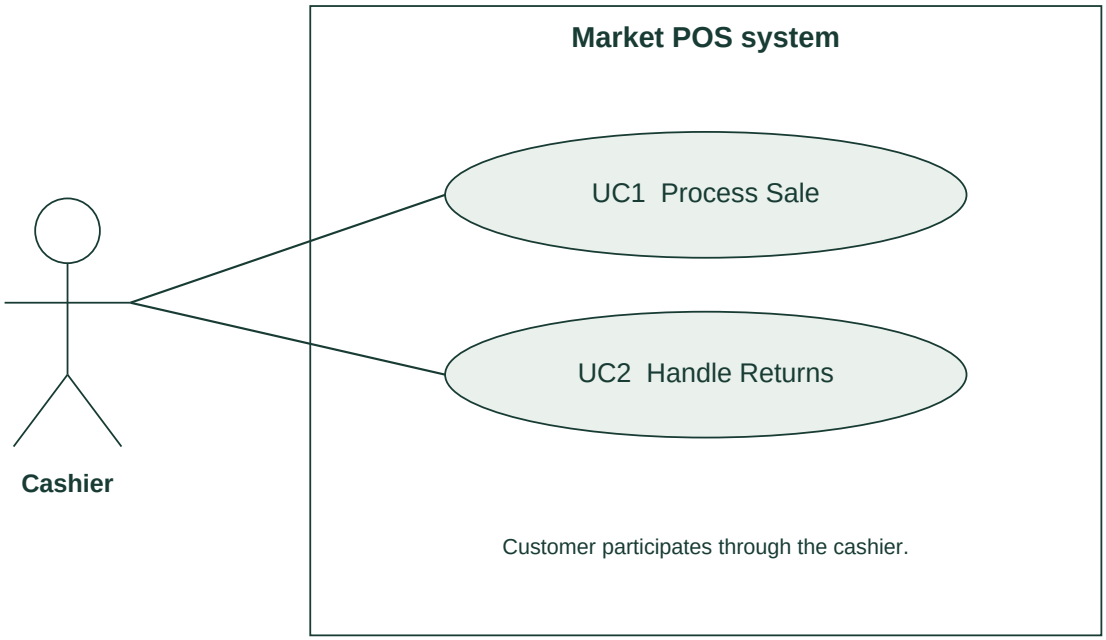
Nonfunctional requirements

Correctness: totals and refunds are exact to the cent. Persistence: completed transactions survive reopening the database. Usability: the application displays English labels, line-item details and actionable errors. Maintainability: HTTP, business values and storage responsibilities are separated. Portability: the application runs with standard Python and a current desktop browser. Security scope: local-only access, parameterized SQL and browser-origin checks; production authentication is excluded.

Traceability

UC1 maps to US01–US04, POSService.checkout and the sale tests. UC2 maps to US05–US07, POSService.find_sale and POSService.return_items. US08 maps to POSService.history and the browser transaction view. US09 and US10 cover deliverables and publication evidence.

6 Use case model



The two use cases are independent cashier goals. No artificial include or extend relation.

Figure 2. Cashier goals within the POS boundary.

The cashier is the primary actor because the cashier directly operates the system to complete the sale or return. The customer is a stakeholder and participates through the cashier. Inventory is internal stored data rather than an external actor. Neither core use case depends on the other at execution time, so no include or extend relation is forced between them.

Following Larman [1, Chapter 6], the detailed texts below carry the behavioral requirements. The diagram provides context and is not a substitute for the success and alternate scenarios.

7 UC1 Process Sale

Field	Specification
Scope and level	Market POS application; user goal
Primary actor	Cashier
Stakeholders	Customer wants correct charges and change. Store operator wants accurate receipts and stock. Cashier wants clear feedback.
Preconditions	Application and product catalogue are available. Cashier has items to sell. No authenticated session is assumed.
Trigger	A customer presents items at checkout.
Minimal guarantee	Invalid checkout makes no committed changes to sales, sale lines or stock.
Success guarantee	One sale and its line items are persisted; payment and change are recorded; stock decreases by sold quantities; receipt is displayed.

Main success scenario

1. The cashier selects a product by name or code and enters its quantity.

2. The system displays the product, unit price, line subtotal and running total.

3. The cashier repeats item entry or adjusts quantities until the basket is complete.

4. The cashier receives cash and enters the amount received.

5. The system rechecks current products and stock, calculates the authoritative total and validates sufficient cash.

6. The system atomically stores the sale and line items, deducts stock and records the request token.

7. The system displays a receipt with sale ID, time, item details, total, cash received and change. The cashier gives the customer the receipt and change.

8 UC1 alternatives and system events

Extensions

At step	Condition and response
1–3	Unknown product or invalid quantity: show an error. The cashier corrects the request. Empty baskets cannot be submitted.
3	The cashier clears the basket before payment: discard the browser draft. No persisted sale exists.
5	Insufficient cash: keep the basket and request a larger amount. No stock is deducted.
5	Insufficient stock or changed availability: reject the sale. Refresh the catalogue and adjust quantities.
6	Storage error: roll back the entire transaction and show a retry message.
6–7	Response is lost: retry the unchanged request with the same token. Return the existing receipt without another deduction.

Special requirements and variations

Cash values have at most two decimal places. Item quantities are integers from 1 to 999. Receipt printing uses the browser print interface. Card payments, discounts, physical barcode devices and tax breakdowns are not implemented. Frequency is not specified by the assignment.



Figure 3. Process Sale system sequence diagram. Basket events are conceptual UI operations; the final submission is POST /api/sales.

9 UC2 Handle Returns

Field	Specification
Scope and level	Market POS application; user goal
Primary actor	Cashier
Stakeholders	Customer wants the correct refund. Store operator wants to prevent duplicate refunds and restore saleable inventory.
Preconditions	The original sale exists. The customer has the receipt ID and goods suitable for restocking.
Trigger	A customer requests a return at checkout.
Minimal guarantee	Rejected return makes no changes to refund records, returned quantities or stock.
Success guarantee	One return and its line items are persisted; returned quantities increase; stock increases; original-price cash refund is displayed.

Main success scenario

1. The cashier enters the original sale ID.

2. The system retrieves original item descriptions, quantities, unit prices and remaining returnable quantities.

3. The cashier selects the items and quantities to return and enters a reason.

4. The system presents a refund preview.

5. The cashier confirms the cash refund.

6. The system rechecks eligibility and atomically records the return, updates returned quantities and restores stock.

7. The system displays a refund receipt. The cashier pays the displayed amount to the customer.

10 UC2 alternatives and system events

At step	Condition and response
1–2	Receipt ID not found: show an error and request correction. Receipt-less returns are outside scope.
3–5	No items selected, invalid quantity or missing reason: reject and allow correction.
6	Item was not purchased or quantity exceeds the remaining amount: reject the whole return.
6	Another request already returned an item: serialized eligibility check prevents over-refunding.
6	Storage error: roll back all return and stock changes.
6–7	Retry of the same token and payload: return the original refund result with no second stock adjustment.

There is no automatic refund to a payment card. The cashier performs the physical cash refund. A later catalogue price change does not change the refund, because the original sale line supplies the unit price. No restocking fee or return deadline is configured.

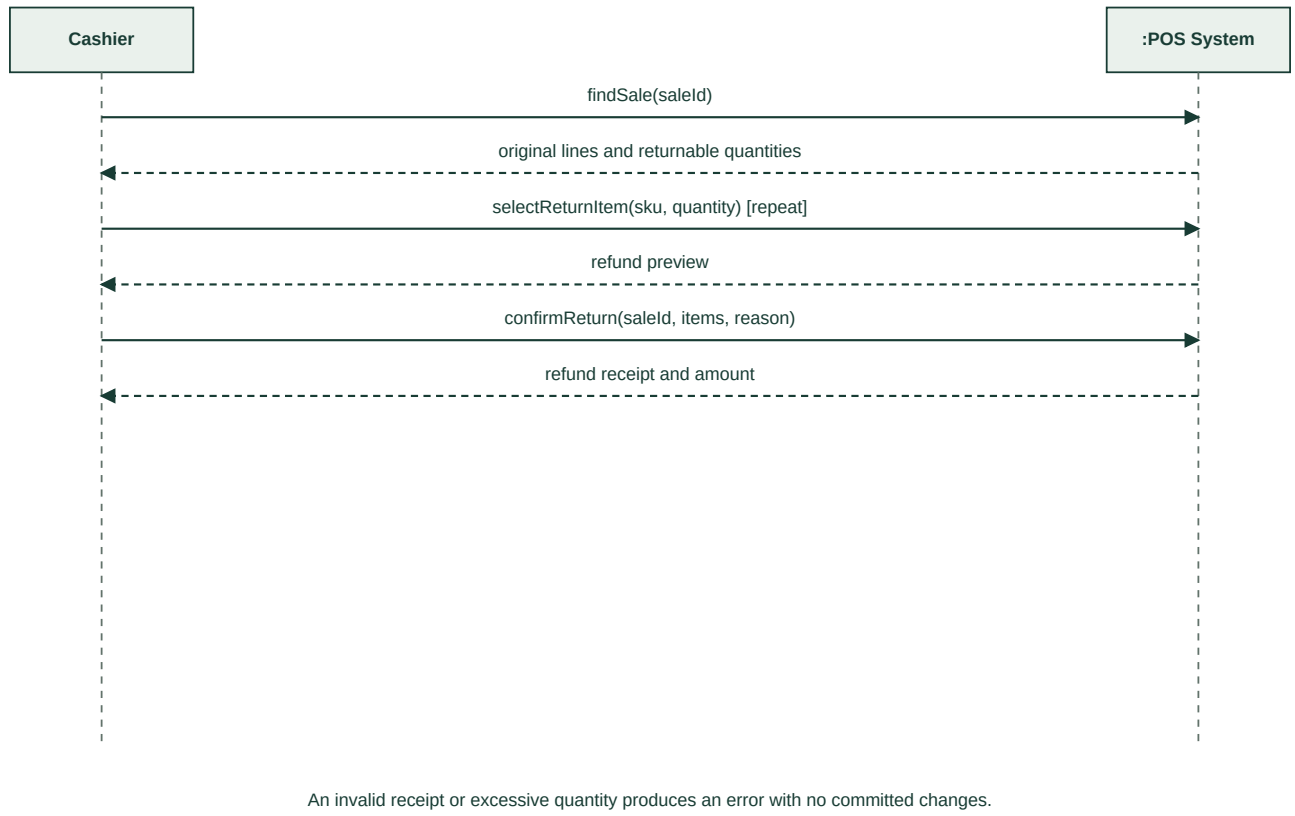
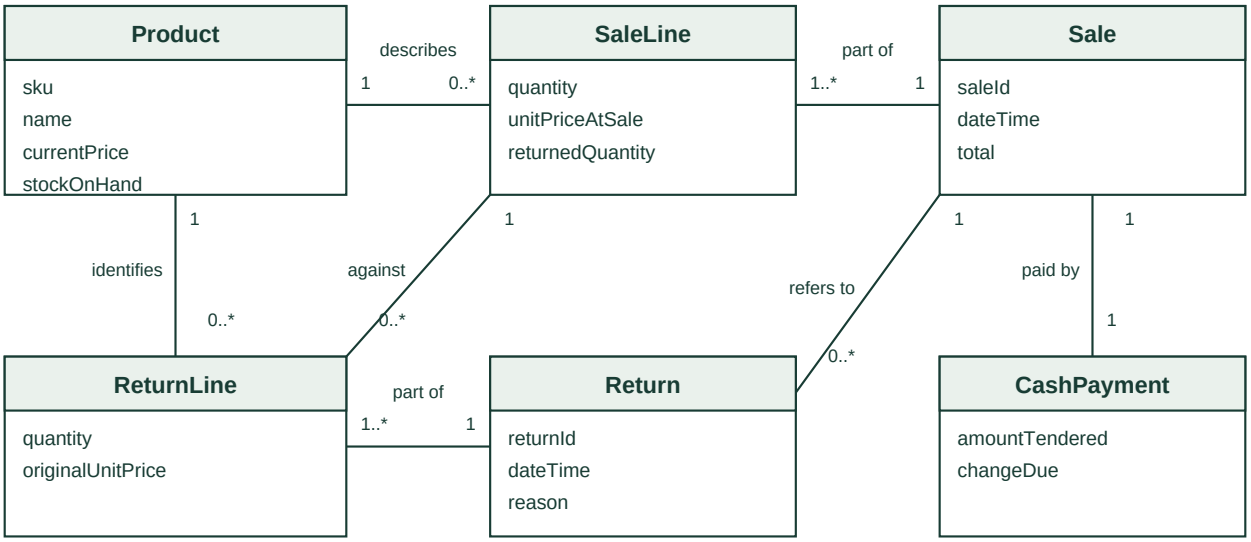


Figure 4. Handle Returns system sequence diagram. Confirmation maps to POST /api/returns.

11 Domain model



Conceptual domain model: business concepts, attributes and multiplicities.

Return and ReturnLine are persisted as records in the implementation.

Figure 5. Conceptual business model with multiplicities. All associations refer to business concepts, not UI widgets or SQL tables.

A Sale contains one or more SaleLines. Each line describes one Product and captures the unit price at purchase. A CashPayment settles one Sale. A Return refers to one original Sale and contains one or more ReturnLines. ReturnLines identify products and correspond to their original SaleLines.

The conceptual model names concepts required for reasoning about the business. The implementation uses immutable Python value objects for Sale, SaleLine and CashPayment. Return and ReturnLine are represented as SQLite rows and dictionaries rather than separate Python classes. This deliberate mapping avoids claiming that every analysis concept has an identical software class.

Key invariants

Sale.total equals the sum of quantity multiplied by original unit price. Product.stock never becomes negative. For each sale line, zero is less than or equal to returned quantity, which is less than or equal to purchased quantity. Refund total equals the returned quantity multiplied by the stored original price.

12 Logical architecture

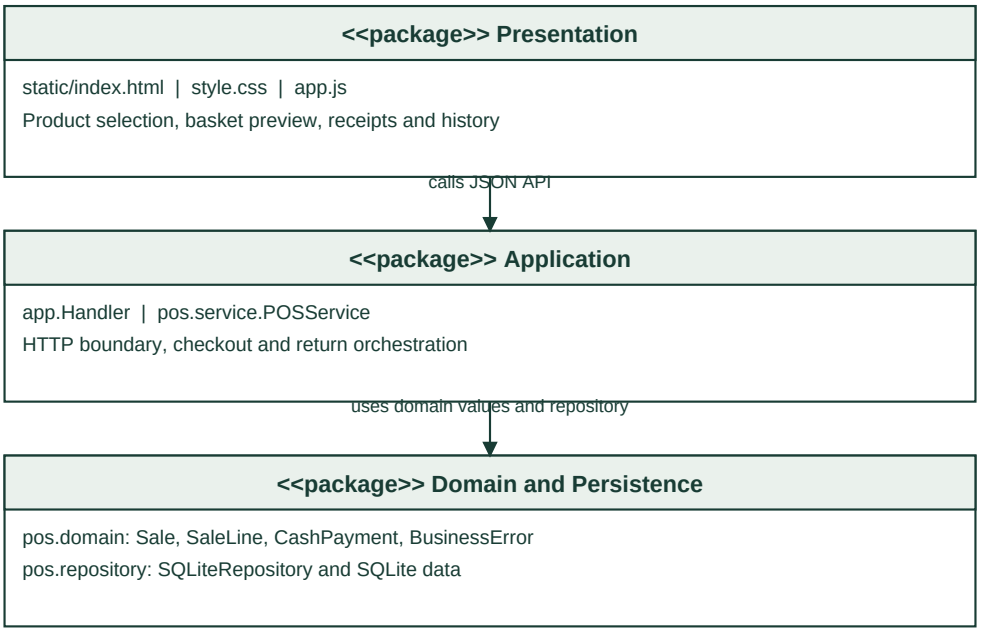


Figure 6. UML package view of the local application.

The presentation package owns product search, draft quantities and preview totals. Its calculations improve usability, but the server independently recomputes values before committing. The application package accepts HTTP requests and coordinates use cases. Domain values perform calculations and validation, while persistence manages database connections and stored records.

The source dependencies are directed: app.py imports POSService and SQLiteRepository; service.py imports domain values and uses the supplied repository; domain.py imports neither HTTP nor SQLite. No database credential or remote service is required. The lower layer contains two distinct modules, which can be separated further if the project grows.

13 Object design and responsibilities

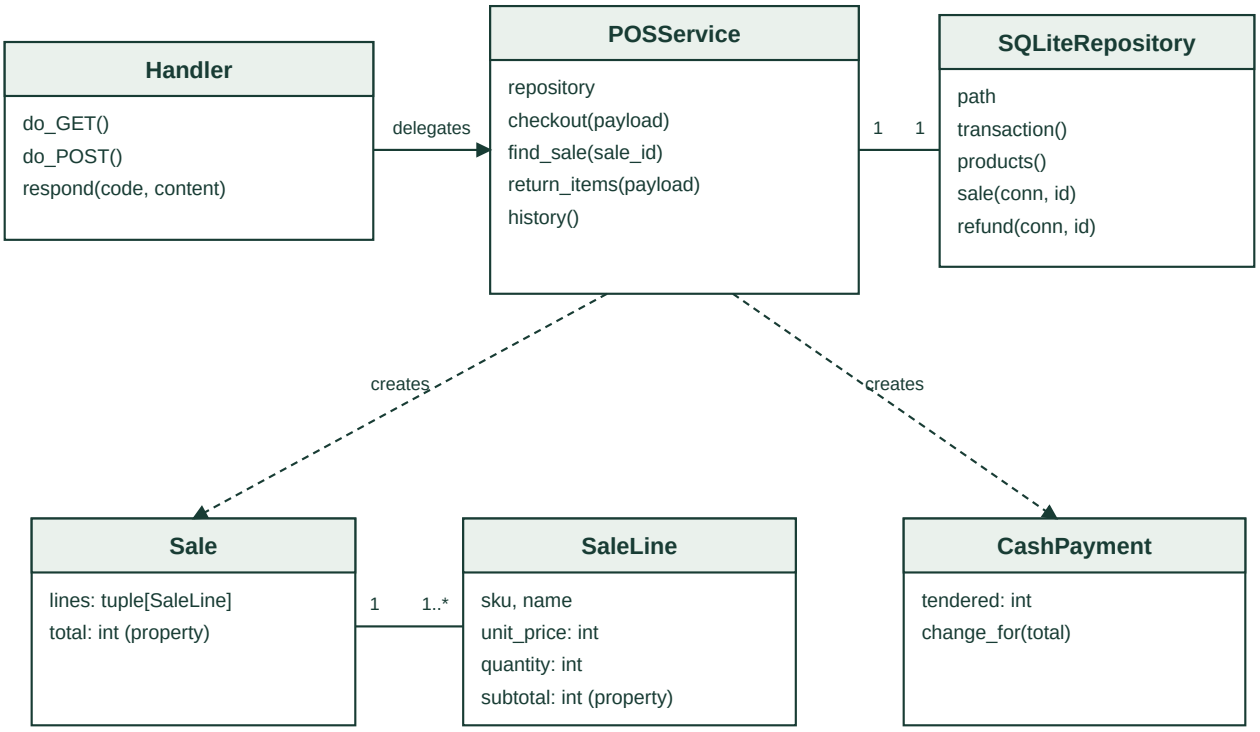


Figure 7. Software classes and selected public operations, consistent with the implementation.

Principle	Application to this project
Controller	POSService receives checkout and return use-case requests, keeping HTTP formatting in Handler.
Information Expert	SaleLine calculates subtotal from its quantity and unit price. Sale sums its own lines. CashPayment validates and calculates change.
High Cohesion	Domain values calculate, repository manages persistence, and browser code presents and collects input.
Low Coupling	The service receives its repository in the constructor, and tests supply a repository backed by a temporary database.

These choices follow the responsibility-driven design discussion in Larman [1, Chapters 17–18]. The repository is a concrete collaborator, not a claimed abstract interface or full dependency-injection framework.

14 Sale interaction design

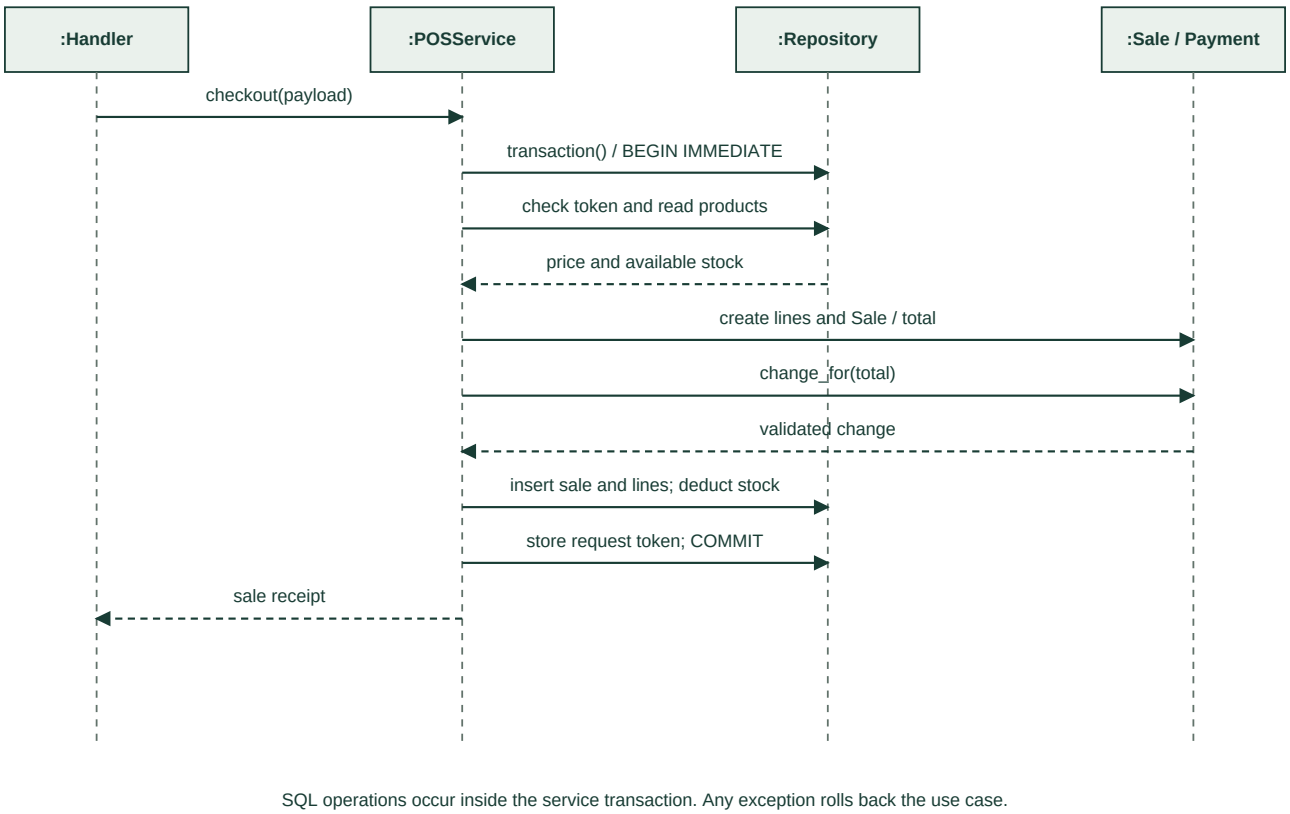


Figure 8. Design sequence for a successful cash checkout.

The handler converts JSON input into a call to checkout. POSService validates line inputs, enters an immediate transaction and checks the request token. It loads authoritative prices and stock, creates SaleLine and Sale values, and asks CashPayment for change. It inserts the sale and lines, deducts stock and stores the token before the transaction commits.

Operation contract

checkout(payload)
Precondition: payload contains items, a cash string and a request token. Postcondition on success: either an existing matching request result is returned or exactly one new sale is created and stock decreases by its quantities. On any business exception or storage exception, all new writes in the transaction roll back.

The sequence diagram groups Sale and CashPayment values on one labeled lifeline for readability. The implementation source distinguishes their separate objects. The repository yields a connection; SQL statements inside the service use that connection within the same transaction.

15 Return interaction design

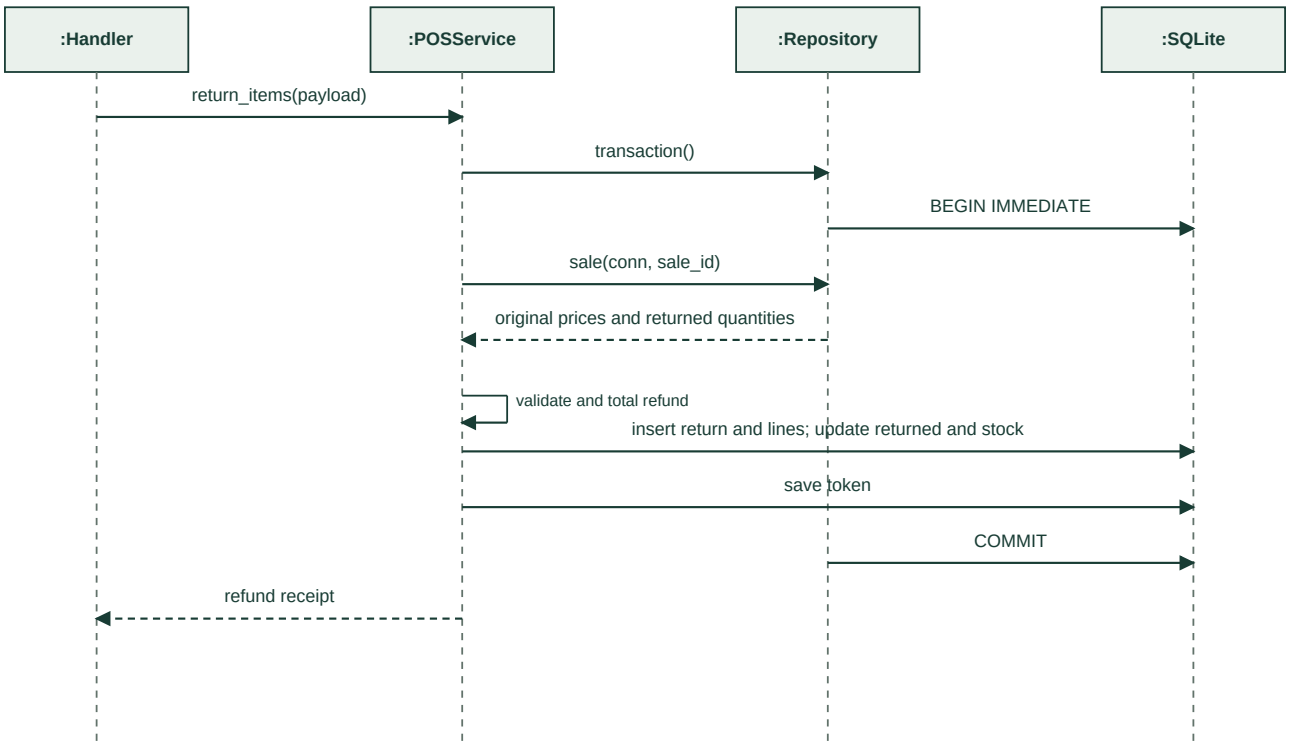


Figure 9. Design sequence for a receipt-based return. The service performs validation and refund calculation.

The service loads the original sale inside the transaction, validates each requested quantity against quantity minus returned, and calculates the refund from original unit prices. Only after every line passes does it insert the return and update stock. A failure on any line prevents partial refund records or partial restocking.

Operation contract

return_items(payload)
Precondition: the receipt exists and the request identifies eligible items, quantities and a reason.
Postcondition on success: one refund record exists, the selected sale-line returned quantities increase, inventory increases by the same quantities, and the refund total equals original-price line totals. A repeated matching request has no additional effects.

16 Persistence and API

Table	Key and purpose
products	sku primary key; catalogue price and non-negative stock
sales	sale ID primary key; timestamp, total, tendered and change
sale_lines	(sale ID, sku) key; original description/price, quantity and returned amount
returns	return ID primary key; original sale ID, timestamp, total and reason
return_lines	(return ID, sku) key; quantity and original price
requests	token primary key; operation kind, payload fingerprint and result ID

Method and endpoint	Result
GET /api/products	Catalogue and current stock
POST /api/sales	Validate and commit a cash sale
GET /api/sales/{saleId}	Original receipt and returnable quantities
POST /api/returns	Validate and commit a receipt-based return
GET /api/history	Latest 100 sales/returns and all-record summary totals

SQLite foreign keys protect record relationships. CHECK constraints protect non-negative stock and bounds on returned quantities. BEGIN IMMEDIATE serializes writers before they read eligibility, preventing two requests from both selling the last unit or refunding the last returnable item. Parameterized SQL separates values from query syntax.

A SHA-256 fingerprint binds an idempotency token to one payload and operation kind. A matching retry returns the existing receipt. Reusing a token for different data is rejected. The token is a duplicate-request mechanism, not an authentication credential.

17 Working cash checkout

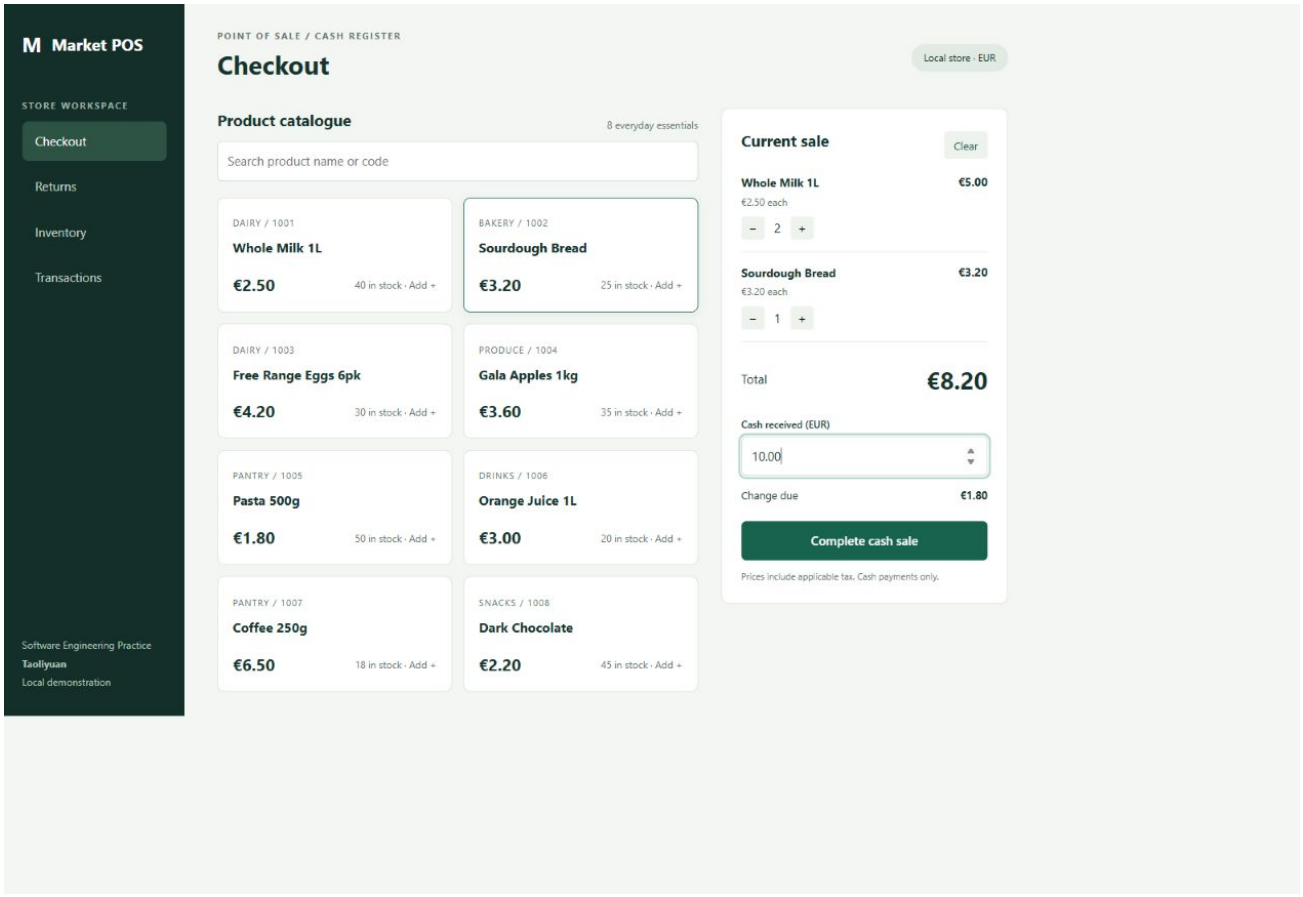


Figure 10. Actual browser basket: two milk units and one loaf cost EUR 8.20. Cash EUR 10.00 previews EUR 1.80 change.

The cashier searches the catalogue or selects a product card. Quantity buttons adjust the draft basket and update its total. Complete cash sale sends product codes, quantities, cash and a request token. Product prices in the browser are previews; the server loads its own catalogue values.

The verified sale ID in the walkthrough was S-346804C9A34C. Completion stored EUR 8.20 total, EUR 10.00 cash and EUR 1.80 change. Milk stock decreased from 40 to 38 and bread stock from 25 to 24. The full receipt screenshot is included in docs/screenshots/sale-receipt.png.

18 Working returns and history

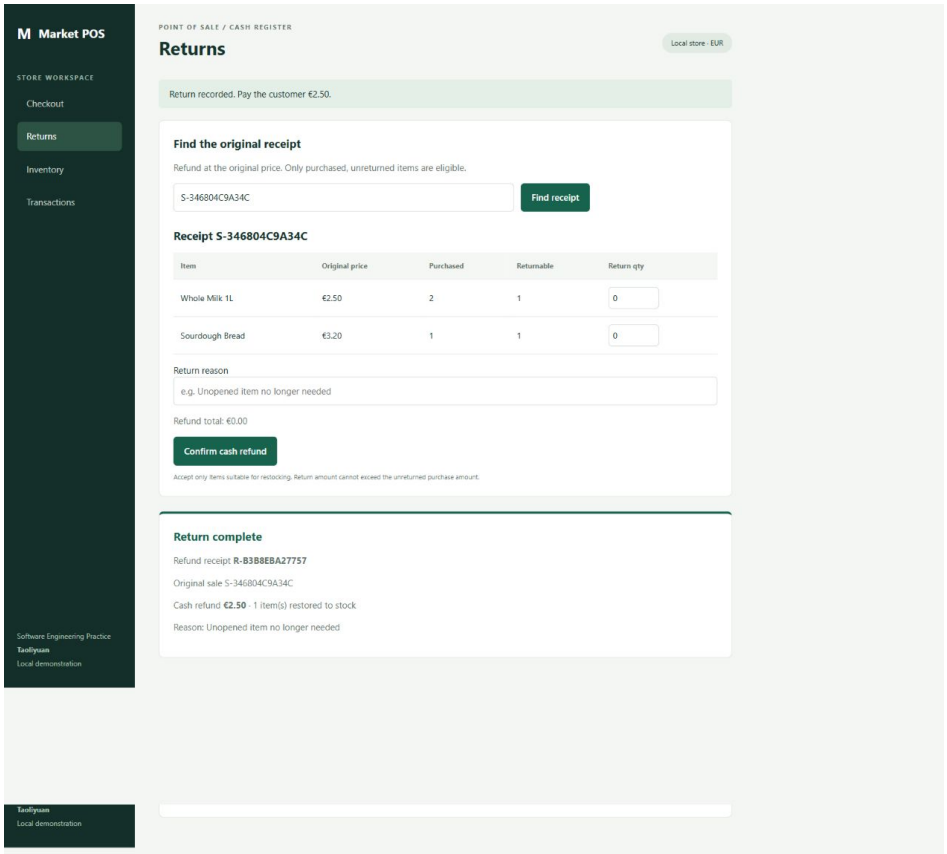


Figure 11. Actual partial return of one milk unit. The refund is EUR 2.50 and the receipt now has one milk unit remaining eligible.

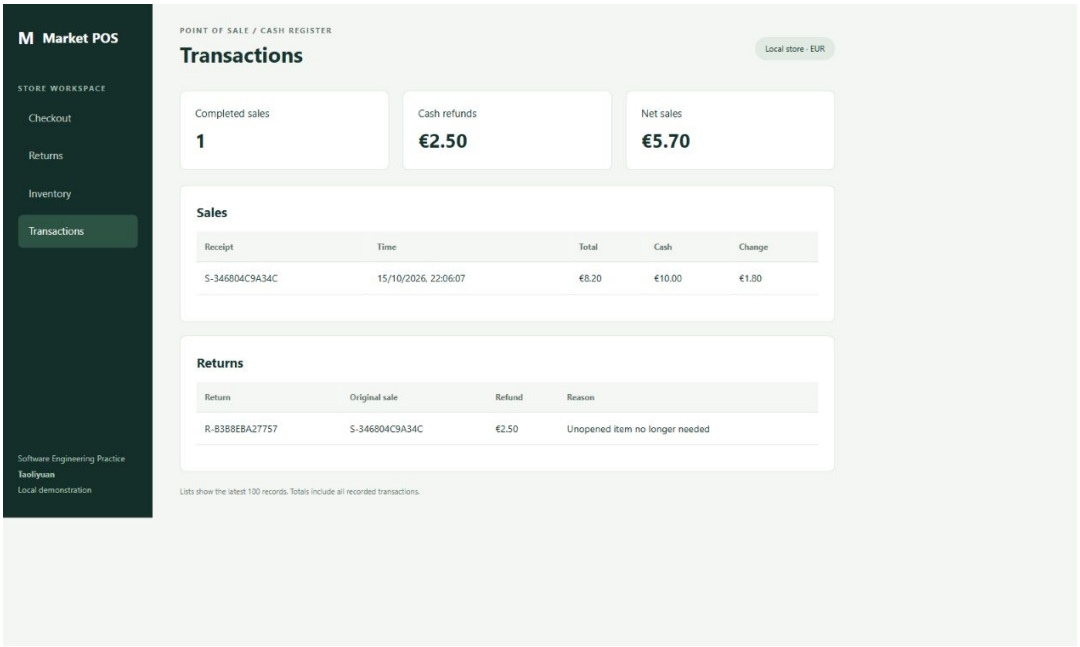


Figure 12. Actual transaction history after the sale and return: net sales EUR 5.70.

19 Verification results

The automated suite ran on 15 October 2026 and passed 22 tests. The command is `python -m unittest discover -s tests -v`. Tests create temporary databases, so the test suite does not change the demonstration database.

Area	Checks and observed result
Cash sales	Correct EUR 8.20 total, EUR 10 tendered, EUR 1.80 change and expected stock deduction.
Invalid requests	Empty basket, invalid quantities, unknown products, invalid money and insufficient cash or stock are rejected.
Atomicity	A rejected sale or mixed invalid return makes no partial stock or transaction changes.
Returns	Partial/full refund, original-price calculation, missing receipt, nonpurchased item and required reason are checked.
Duplicate requests	Identical sale/return retry returns the existing ID; changed payload with an old token fails.
Persistence	A new repository instance reads the stored sale and stock correctly.
Concurrency	Two requests competing for the last unit produce one success. Two returns competing for the last eligible quantity produce one success.
HTTP boundary	Catalogue and checkout work. A non-object JSON request returns 400 and an unrelated browser origin returns 403.

Manual browser acceptance

The application was exercised through the actual browser interface: basket creation, payment, receipt retrieval, partial refund, stock view and transaction history. Screenshots preserve the observable results. The automated suite provides error-path evidence. Neither this test result nor the browser walkthrough claims a load test, penetration test or student acceptance review.

Known limitations

Draft baskets are lost on refresh. History lists are limited to 100 records, although summaries cover all transactions. The local development server, lack of authentication and cash-only workflow make deployment scope intentionally narrow. No printing hardware or card provider was tested.

20 Version control and delivery

The repository began with a planning commit on main. Implementation proceeded on taoliyuan/development. The development branch is retained when its completed increment is merged to main. Local commit identity uses the supplied project owner, with AI assistance disclosed in the README. Timestamps are actual execution times.

Commit	Purpose
f2811d3	Plan individual POS scope and acceptance criteria
b44e312	Implement atomic cash sales, receipt returns and English interface
314bd47	Verify workflows with 22 tests and browser evidence

The complete current commit graph and commands are supplied in docs/git-log.txt and docs/GIT_COMMAND_HISTORY.md. Later documentation and merge commits appear there. Remote branch verification is recorded separately after a successful push.

Submission components

Deliverable	File or destination
Technical report	Technical_Report.pdf
Project source and tests	ProjectPos_2026 directory and source archive
Agile plan and burndown	Agile_Plan.xlsx
Presentation	POS_Presentation.pptx
English narration and recording steps	docs/DEMO_SCRIPT.md
GitHub repository	https://github.com/Welt-Tao/ProjectPos_2026 — Published; main and taoliyuan/development verified on 15 October 2026
Presentation video	Pending student recording and publication

The presentation should explain the slides and switch to the running system for both workflows. The student records the English narration, uploads to Bilibili and verifies the public viewing URL. The report cover is updated with that URL after publication.

21 Evaluation and references

Evaluation

The implemented scope satisfies the two central POS workflows. The design preserves original purchase prices, prevents negative stock and excessive refunds, and connects the analysis models to code and tests. The strongest technical evidence is the combination of a successful real interface walkthrough and tests of failure, retry and concurrency behavior.

An individual Scrum-inspired workflow provides a clear backlog and acceptance checks, but cannot demonstrate team collaboration. Planned multi-day sprints and actual accelerated implementation are therefore represented separately. The student review, recording and repository publication complete the remaining delivery work.

Future development

Useful extensions include authenticated cashier accounts, permission-controlled stock adjustments, damaged-goods return handling, receipt printer integration and card payment through a verified provider. Any production deployment would require a production server, access controls, backups, operational monitoring and additional security tests.

References

[1] Craig Larman. Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development, third edition. Prentice Hall, 2005. Supplied course PDF. Relevant chapters: 6 Use Cases; 9 Domain Models; 10 System Sequence Diagrams; 13 Logical Architecture and UML Package Diagrams; 15 UML Interaction Diagrams; 16 UML Class Diagrams; 17–18 GRASP and Object Design.

[2] Software Engineering Practice Assignment. Supplied course assignment document. Requirements for Agile planning, Process Sale, Handle Returns, UML, GitHub and final submission.

[3] Tech Report Cover Page. Supplied course cover template. Institutional artwork and cover field structure retained for this report.

Source availability

Complete executable source is included in the accompanying ProjectPos_2026 folder and archive. The next appendix provides the central application controller listing. The domain, persistence, interface and tests are available as separate readable source files in the project.

Appendix A Application controller

The following is the executable POSService module. See pos/domain.py for money and quantity validation and pos/repository.py for transaction handling and constraints.

```
"""Application controller for sale and return use cases."""
import hashlib
import json
import uuid
from datetime import datetime, timezone
from .domain import BusinessError, Sale, SaleLine, CashPayment, money, quantity


class POSService:
    def __init__(self, repository):
        self.repository = repository

    @staticmethod
    def _lines(items):
        if not isinstance(items, list) or not 1 <= len(items) <= 100:
            raise BusinessError("Select at least one item, up to 100 lines.")
        merged = {}
        for item in items:
            if not isinstance(item, dict) or not isinstance(item.get("sku"), str):
                raise BusinessError("Each item needs a product code and quantity.")
            sku = item["sku"]
            merged[sku] = quantity(merged.get(sku, 0) + quantity(item.get("quantity")))
        return merged

    @staticmethod
    def _request(conn, payload, kind):
        token = payload.get("token")
        if not isinstance(token, str) or not 8 <= len(token) <= 100:
            raise BusinessError("A valid request token is required.")
        fingerprint = hashlib.sha256(json.dumps(payload, sort_keys=True).encode()).hexdigest()
        old = conn.execute("SELECT * FROM requests WHERE token=?", (token,)).fetchone()
        if old and (old["kind"] != kind or old["fingerprint"] != fingerprint):
            raise BusinessError("This request token was already used for different data.")
        return token, fingerprint, old["result_id"] if old else None

    def checkout(self, payload):
        items = self._lines(payload.get("items"))
        payment = CashPayment(money(payload.get("cash")))
        with self.repository.transaction() as conn:
            token, fingerprint, old_id = self._request(conn, payload, "sale")
            if old_id:
                return self.repository.sale(conn, old_id)
            lines = []
            for sku, count in items.items():
                product = conn.execute("SELECT * FROM products WHERE sku=?", (sku,)).fetchone()
                if product is None:
                    raise BusinessError(f"Unknown product code: {sku}.")
                if count > product["stock"]:
                    raise BusinessError(f"Not enough stock for {product['name']}.")
                lines.append(SaleLine(sku, product["name"], product["price"], count))
            sale = Sale(tuple(lines))
            change = payment.change_for(sale.total)
            sale_id = "S-" + uuid.uuid4().hex[:12].upper()
            conn.execute("INSERT INTO sales VALUES (?,?,?,?,?)", (sale_id,
    datetime.now(timezone.utc).isoformat(), sale.total, payment.tendered, change))
            for line in sale.lines:
                conn.execute("INSERT INTO sale_lines VALUES (?,?,?,?,?,0)", (sale_id, line.sku,
    line.name, line.unit_price, line.quantity))
                conn.execute("UPDATE products SET stock=stock-? WHERE sku=?", (line.quantity,
    line.sku))
            conn.execute("INSERT INTO requests VALUES (?,?,?,?)", (token, "sale", fingerprint,
    sale_id))
            return self.repository.sale(conn, sale_id)

    def find_sale(self, sale_id):
        with self.repository.transaction() as conn:
```

Appendix A Controller continued

```
063             result = self.repository.sale(conn, sale_id)
064             if not result:
065                 raise BusinessError("Receipt not found. Check the sale ID.")
066             return result
067 
068     def return_items(self, payload):
069         items = self._lines(payload.get("items"))
070         sale_id = payload.get("sale_id")
071         reason = payload.get("reason", "")
072         if not isinstance(sale_id, str) or not isinstance(reason, str) or not 3 <=
    len(reason.strip()) <= 200:
073             raise BusinessError("Provide a sale ID and a return reason of 3 to 200 characters.")
074         with self.repository.transaction() as conn:
075             token, fingerprint, old_id = self._request(conn, payload, "return")
076             if old_id:
077                 return self.repository.refund(conn, old_id)
078             sale = self.repository.sale(conn, sale_id)
079             if not sale:
080                 raise BusinessError("Receipt not found. Check the sale ID.")
081             original = {line["sku"]: line for line in sale["lines"]}
082             total = 0
083             for sku, count in items.items():
084                 if sku not in original:
085                     raise BusinessError("This product is not on the original receipt.")
086                 line = original[sku]
087                 if count > line["quantity"] - line["returned"]:
088                     raise BusinessError(f"Return quantity exceeds the remaining quantity for
    {line['name']}.")
089                 total += count * line["unit_price"]
090             return_id = "R-" + uuid.uuid4().hex[:12].upper()
091             conn.execute("INSERT INTO returns VALUES (?,?,?,?,?)", (return_id, sale_id,
    datetime.now(timezone.utc).isoformat(), total, reason.strip()))
092             for sku, count in items.items():
093                 conn.execute("INSERT INTO return_lines VALUES (?,?,?,?)", (return_id, sku, count,
    original[sku]["unit_price"]))
094                 conn.execute("UPDATE sale_lines SET returned=returned+? WHERE sale_id=? AND sku=?",
    (count, sale_id, sku))
095                 conn.execute("UPDATE products SET stock=stock+? WHERE sku=?", (count, sku))
096             conn.execute("INSERT INTO requests VALUES (?,?,?,?)", (token, "return", fingerprint,
    return_id))
097             return self.repository.refund(conn, return_id)
098 
099     def history(self):
100         with self.repository.transaction() as conn:
101             return {
102                 "sales": [dict(r) for r in conn.execute("SELECT * FROM sales ORDER BY created_at
    DESC LIMIT 100")],
103                 "returns": [dict(r) for r in conn.execute("SELECT * FROM returns ORDER BY created_at
     DESC LIMIT 100")],
104                 "summary": dict(conn.execute("SELECT (SELECT COUNT(*) FROM sales) AS sale_count,
    (SELECT COALESCE(SUM(total),0) FROM sales) AS gross, (SELECT COALESCE(SUM(total),0) FROM returns) AS
     refunds").fetchone()),
105             }
```